

Documentation for Surface Table Structure

This documentation outlines the current structure of the surface table, the purpose of its columns, and plans for future enhancements. The goal is to create a scalable and efficient system that supports finite element analysis (FEA) and can be extended as the project evolves.

Current Surface Table Structure

The surface table currently includes the following columns:

Column	Description	Purpose
local_id	Unique identifier for each surface.	Ensures every surface is uniquely identifiable.
node_1	Node ID for the first corner of the surface.	Defines the surface geometry.
node_2	Node ID for the second corner of the surface.	Defines the surface geometry.
node_3	Node ID for the third corner of the surface.	Defines the surface geometry.
node_4	Node ID for the fourth corner of the surface (or NaN for triangles).	Defines the surface geometry.
stringer_1	Stringer index corresponding to node_1.	Tracks surface alignment with structural elements.
stringer_2	Stringer index corresponding to node_2.	Tracks surface alignment with structural elements.
rib_1	Rib index corresponding to node_1.	Tracks surface alignment with structural elements.
rib_2	Rib index corresponding to node_2.	Tracks surface alignment with structural elements.
tags	Tag describing the surface type (quad, tri, irregular).	Facilitates categorization and downstream processing.

Column	Description	Purpose
area	Precomputed area of the surface.	Supports validation and mesh quality analysis.
aspect_ratio	Aspect ratio of the surface (max/min diagonal for quads or edges for triangles).	Ensures surface quality.

Current Implementation Notes

1. Simplified Structure:

- Focuses on essential columns for geometry definition and quality analysis (area and aspect ratio).
- Tags help categorize surfaces (e.g., quad, tri) for specialized handling.

2. Quality Validation:

- Surface area and aspect ratio are included to detect and avoid degenerate or poorly shaped surfaces.
-

Planned Enhancements

In the future, the surface table can be extended to include the following columns to better support FEA workflows and integration with tools like Patran/Nastran:

1. Connectivity to Adjacent Surfaces

- Columns: adjacent_surface_1, adjacent_surface_2, ...
- Purpose: Tracks adjacent surfaces for connectivity validation and boundary continuity.
- Example: adjacent_surface_1 stores the local_id of a neighboring surface that shares an edge.

2. Material Properties

- Column: material_id.
- Purpose: Links surfaces to material properties for analysis (e.g., stiffness, density).

3. Element Type

- **Column:** `element_type`.
- **Purpose:** Specifies the finite element type (e.g., CQUAD4, CTRIA3).

4. Boundary Conditions

- **Columns:** `bc_edge_1`, `bc_edge_2`, ...
- **Purpose:** Tracks fixed or constrained edges for boundary condition application.

5. Thickness

- **Column:** `thickness`.
- **Purpose:** Defines shell element thickness for structural analysis.

6. Export Compatibility

- **Columns specific to Patran/Nastran format for easier export and simulation setup.**
- **Example:** `patran_element_id`.

Rationale for Enhancements

1. Connectivity to Adjacent Surfaces:

- Ensures continuity in the mesh and prevents gaps or overlaps in the finite element model.

2. Material Properties and Thickness:

- Enables simulation-ready surfaces with all physical properties defined.

3. Boundary Conditions:

- Facilitates direct mapping of constraints during FEA setup.

4. Export Compatibility:

- Reduces preprocessing effort when transitioning to Patran/Nastran or similar tools.

Steps for Future Implementation

1. Current Implementation:

- Ensure accurate calculation of area and aspect_ratio.
- Validate surfaces during creation using these metrics.

2. Future Implementation:

- Develop a connectivity algorithm to populate adjacent surface columns.
- Extend table structure to include material properties, element type, and other attributes.
- Integrate Patran/Nastran export functionality.

Penta Surfaces (5-Point Surfaces)

Penta surfaces represent surfaces defined by five nodes. These surfaces are captured in the same way as triangular and quadrilateral surfaces, with the addition of a fifth node. Below is the structure and column description for the penta surfaces table:

Table Structure

Column Name Description

local_id	Unique identifier for the penta surface within the dataset.
node_1	Local ID of the first node defining the surface.
node_2	Local ID of the second node defining the surface.
node_3	Local ID of the third node defining the surface.
node_4	Local ID of the fourth node defining the surface.
node_5	Local ID of the fifth node defining the surface.
stringer_1	Associated stringer index for the first edge of the surface.
stringer_2	Associated stringer index for the second edge of the surface.
rib_1	Rib index associated with the first boundary of the surface.
rib_2	Rib index associated with the second boundary of the surface.
tags	Tags describing surface type, e.g., "penta irregular".
area	Area of the surface in square units.

Column Name Description

aspect_ratio Aspect ratio of the surface for geometry verification.

Notes on Usage

- The node_5 column allows for flexibility in capturing geometries that require five defining points.
- FEA Compatibility: If analysis software such as Patran does not support 5-point surfaces, these can be split into two quadrilateral surfaces, as described in the section on surface division.
- Tags: The tags column should specify the nature of the surface, e.g., "penta irregular" or "penta inserted," to differentiate it from other surface types.

Example

Below is an example of a penta surface table entry:

local	node	node	node	node	node	string	string	rib	rib		are	aspect_r
_id	_1	_2	_3	_4	_5	r_1	r_2	_1	_2	tags	a	atio
101	10	15	20	25	30	1	2	-1	-2	"penta irregular"	12.34	1.56

Handling in MATLAB

In MATLAB, penta surfaces are captured similarly to triangular and quadrilateral surfaces, using a table structure. The additional node_5 is simply appended as a column in the existing workflow.

Export Considerations

- Patran: If 5-point surfaces are unsupported, the surfaces should be divided into two 4-point surfaces using a dedicated MATLAB function.
- Solver Compatibility: Ensure that solvers used in the analysis can handle 5-point surfaces or adjust accordingly.